Evaluation Report of EliteFS-Day47-LossFunctions-KMIT-KMIT

Average Score: 8/10 | Attempted: 3 of 3 questions

Question 1

THEORY EVALUATE Best Score: 8/10

When would you prefer Huber Loss over both MSE and MAE in a real-world application?

Desired Answer

Huber Loss is ideal when the dataset contains some outliers but not too many. It behaves like MSE for small errors, ensuring smooth optimization, and like MAE for large errors, reducing sensitivity to outliers. This balance makes it more stable in practice. It is widely used in regression tasks such as finance or robotics. It provides a good trade-off between robustness and convergence speed.

Student's Answer

Huber Loss is preferred over MSE and MAE because it has a delta term that determines if the loss is lying >delta then MAE would be considered.. as it provides absolute difference of errors..Whereas when the loss error is less than the delta term then, MSE(Mean Squared Error) is considered, as the losses are small, its square would not create much impact.. If we use MSE for large errors, eg: in outliers in that case the overall loss would be so high.. therefore a balance of MSE and MAE is essential.

Evaluations

Feedback 1

★★★★☆ 8/10

The student is demonstrating understanding of the same concepts as the Actual Answer: they explain that Huber Loss uses a delta threshold to switch between MSE for small errors and MAE for large errors, and note that this balances sensitivity to outliers versus smooth optimization. This matches the core idea that Huber Loss behaves like MSE for small residuals and like MAE for large residuals, making it robust to outliers but still smoothly optimizable. They also reference outliers and the need for this balance. Where the student falls slightly short is in articulating the typical real-world contexts (e.g., finance or robotics) and explicitly mentioning the improved convergence speed and robustness trade-off. Overall, however, they accurately capture the mechanism and rationale behind preferring Huber Loss, warranting a score in the 'Good Understanding' range.

Feedback 2

★★★★☆ 8/10

The student is demonstrating understanding of the same concepts as the Actual Answer. They correctly describe the delta threshold in Huber Loss, explain that for errors below delta it behaves like MSE and for errors above delta it switches to MAE, and note that this balances sensitivity to outliers vs. smooth optimization. These points align with the Actual Answer's core: Huber Loss's hybrid behavior, reduced sensitivity to large errors, and stability. The response, however, omits mention of typical application domains (e.g., finance or robotics) and the trade-off in convergence speed, which are secondary details in the Actual Answer. Overall, the student shows a solid grasp of the main mechanism and rationale, but misses some context and secondary motivations, justifying an 8/10.

Question 2

THEORY UNDERSTAND Best Score: 8/10

Why does Binary Cross Entropy produce strong gradients for wrong predictions, and how does this help training?

Desired Answer

Binary Cross Entropy penalizes predictions that are confidently wrong using the logarithmic function. When the predicted probability is far from the true label, the loss increases sharply. This results in strong gradients that push the model to correct its mistakes quickly. It accelerates learning in classification tasks. This behavior makes BCE effective for binary classification problems.

Student's Answer

Binary Cross Entropy follows logarithmic functions and it produces strong gradients for wrong predictions.. Lets say model predicted 1(true) but actual label is 0. In that case: -[y*log(P)+(1-y)log(p)]=>-[1*log(0)]=>-[approaches(-infinity)]=>infinity.. Therefore the loss is huge... This helps in training by penalizing the weights that do not contribute to the output.

Evaluations

Feedback 1

7/10

The student is demonstrating understanding of the same concepts as the Actual Answer: that BCE uses a logarithmic penalty to produce large losses (and hence strong gradients) for confident but wrong predictions. They correctly show via the formula that predicting 1 when the true label is 0 leads to $\log(0) \to -\infty$ and thus an infinite loss. They also note this 'huge' loss penalizes the model weights. What they miss is an explicit statement that these strong gradients accelerate correction of mistakes and speed up learning in binary classification tasks. Instead, they vaguely say it penalizes 'weights that do not contribute to the output' without tying this back to faster convergence. Overall, they capture the core mechanism (log penalty → large loss → strong gradients) but only implicitly touch on how this helps training, so the explanation is solid but not fully complete.

Feedback 2

8/10

The student is demonstrating understanding of the same concepts as the Actual Answer. They correctly explain that BCE uses a logarithmic function, show via the example of predicting 1 when the true label is 0 that log(0) drives the loss toward infinity, and note that this large loss (and corresponding strong gradient) penalizes the model's weights. They capture why confident wrong predictions incur heavy penalties and how this influences training by pushing the model to adjust its parameters. They omit an explicit statement about accelerating learning or class-specific effectiveness, but overall their answer reflects a solid grasp of the key mechanism and its training benefit.

Question 3

THEORY UNDERSTAND Best Score: 7/10

Why is Mean Squared Error (MSE) highly sensitive to outliers, and in which scenarios would this be a disadvantage?

Desired Answer

MSE squares the error term, so larger errors get amplified significantly compared to smaller ones. This makes the model focus heavily on outliers rather than overall trends. In noisy datasets or real-world measurements (like sensor data), this can distort learning. It is disadvantageous when outliers are not meaningful or are due to noise.

Student's Answer

MSE(Mean Squared Error) are highly sensitive to outliers, if the difference between actual and predicted is large, the square is ought to be quadratically large.. It is highly sensitive to outliers.. In scenarios like when the data is highly sparse, where few data points are very far away from each other then, it might be problematic.

Evaluations

Feedback 1

7/10

The student is demonstrating understanding of the same concepts as the Actual Answer: they correctly explain that squaring the error term amplifies large deviations (outliers) and identify a scenario where this is problematic (when data contains a few very distant points). This aligns with the core explanation that MSE's quadratic penalty focuses excessively on outliers. However, their discussion of disadvantageous scenarios is narrower than the Actual Answer's coverage of noisy or spurious measurements (e.g., sensor noise). The answer is accurate and on-point but lacks the broader illustration of real-world noise contexts, so it earns a good (7) for clear but somewhat limited understanding.

Feedback 2

7/10

The student is addressing the same concepts as the Actual Answer: they explain that MSE squares the error term, causing large errors (outliers) to be amplified, and they identify a scenario (sparse data with widely separated points) where this sensitivity is problematic. This shows a clear grasp of why MSE is sensitive to outliers. They miss the specific emphasis on noisy measurements or meaningless outliers, and they do not mention that this focus on outliers can distort learning of overall trends. Thus, their understanding is solid but not fully complete, meriting a score of 7.